RoamPredict

Report: Part 3, Act and Remember

How a switch is carried out safely, how the network layer is checked, and how every outage is measured and recorded.

Author: Adil · Code version: commit 82b4b4d (branch claude/upbeat-newton-gh5kgn) · 264 automated tests

Summary

RoamPredict predicts that a laptop's Wi-Fi link is about to fail and asks Windows to move to a better access point (AP) early. Parts 1 and 2 decide when to move and where to. **Part 3 does the move and measures the result.** It has two jobs:

- **Act**: carry out a switch safely, check that the internet works afterwards, and give control back to Windows when anything goes wrong.
- **Remember**: measure every internet outage, store every switch and outage in a SQLite database, and write every event to a log file. This is what lets us compare walks with and without RoamPredict.

Main results: every switch goes through the same verify → connect → check → fallback sequence; the password is never handled by Python; a field test exposed a switch that left the laptop without an IP address, and Part 3 now hands control back to Windows in that case.

Contents

1. Files owned by Part 3
2. Architecture
3. Act: carrying out a switch (executor.py)
4. Act: checking the network layer (l3.py)
5. Act: talking to Windows in C (win32_structs.py)
6. Remember: measuring internet outages (probe.py, latency.py)
7. Remember: the database (store.py)
8. Remember: the log file (telemetry.py)
9. Lists and dictionaries in Part 3
10. Field results and the bug we fixed
11. Limits
12. Conclusion

1. Files owned by Part 3

File	Lines	Role
roam/executor.py	463	Execution pipeline: fresh scan verification, connection request, IP check, fallback
net/l3.py	241	Windows networking: IP address and gateway, ping, ARP flush (FlushIpNetTable2), DHCP renew (ipconfig /renew)
net/probe.py	177	Background internet probe and outage detection
net/latency.py	371	The TCP probe itself, latency statistics, closing RoamPredict's own stale sockets
topology/store.py	570	SQLite database roampredict.db: switches, outages, associations, learned moves
telemetry.py	114	Structured JSON event log roampredict.jsonl, with secrets removed
platform/win32_structs.py	451	Low-level C structures for wlanapi.dll and iphlpapi.dll

All paths are inside the `roampredict` folder. Part 3 is called by the engine (`engine.py`), which owns the main loop.

2. Architecture

Part 3 starts when the engine has decided to switch (state ROAM_IMMINENT, a candidate at least 8 dB stronger, every safety rule satisfied). The simple view:

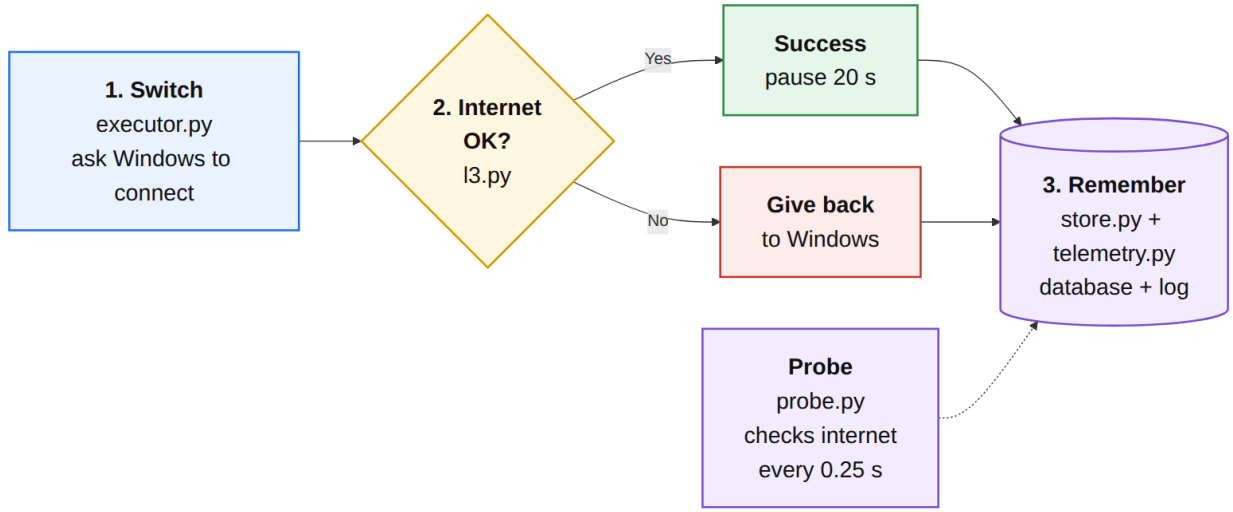


Figure 1. Part 3 in one picture: switch, check, success or give back, remember.

The detailed view shows which object calls which. The engine is the conductor; it never talks to Windows itself.

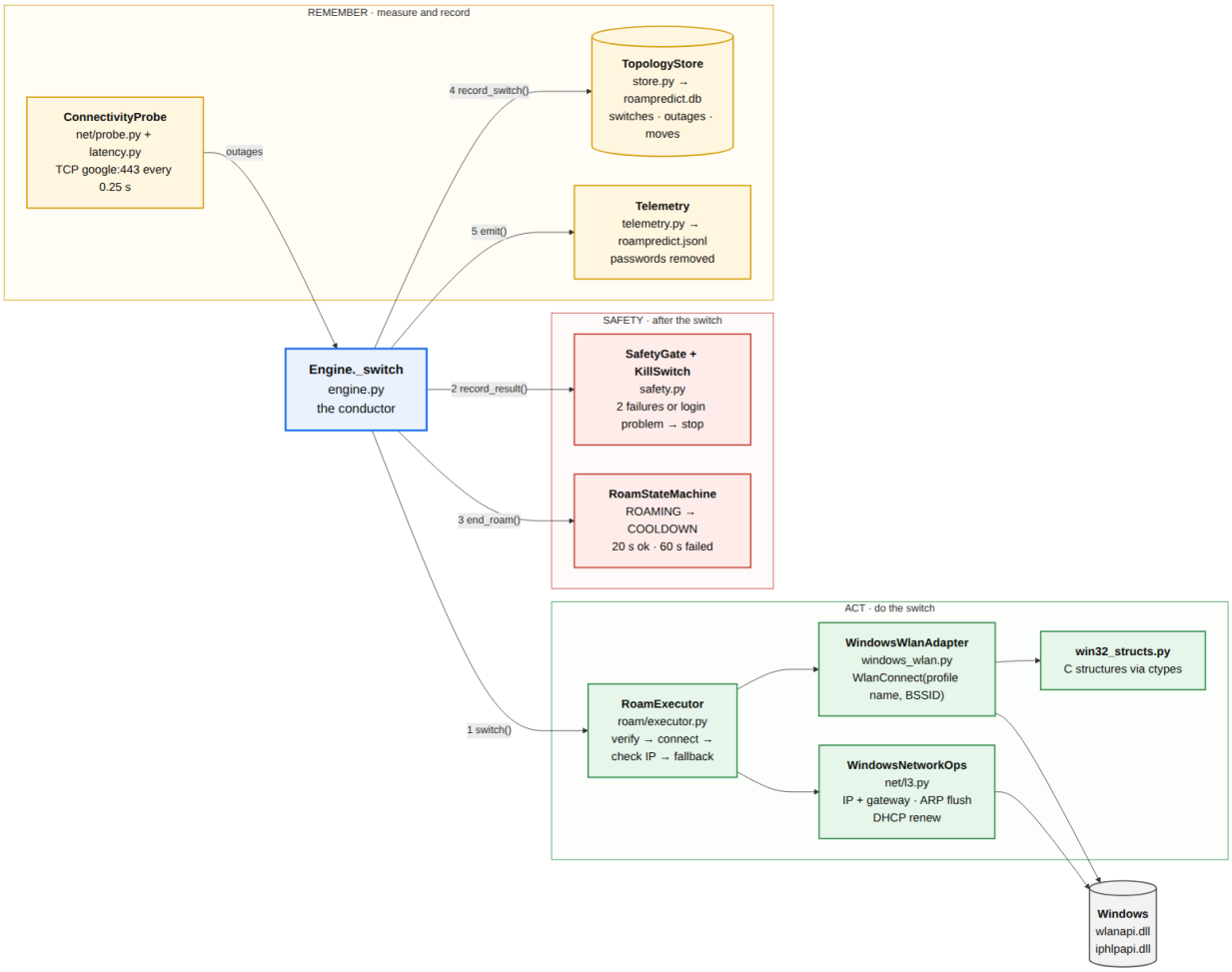


Figure 2. Part 3 components. Arrows are calls, numbered in the order the engine makes them.

Design choices: the executor receives the Wi-Fi adapter and the network layer as parameters (dependency injection), so the tests replace Windows with a fake and run on any computer; the probe runs in its own thread but only puts results in a queue, so all decisions stay on one thread; the database is reached only through `TopologyStore`, which hides the SQL.

3. Act: carrying out a switch (executor.py)

Windows has no public “roam now” command. The only tool is `WlanConnect`: “connect with this saved profile, and I would like this BSSID”. Windows usually drops the old link and connects again, so every switch costs a short break (2 to 7 s measured). The executor therefore follows a strict sequence.

Step	What happens	If it fails
1. Verify	Re-read Windows' AP list; if the candidate was not heard in the last 3 s, request a scan and wait (max 4.5 s). Check it is still visible, the link did not change, and it is still 8 dB stronger (3 dB if the internet is already dead).	NOT_VERIFIED: no switch
2. Connect	WlanConnect with the saved profile's **name** and one desired BSSID. Windows events time each phase: association, 802.1X authentication, connected. Timeout 12 s.	see step 3

Step	What happens	If it fails
3. Connection failed	802.1X stalled or failed: a login window might appear, so stop, leave it to Windows and trip the kill switch. Any other failure: give Windows 5 s to reconnect by itself, then reconnect by profile name.	fallback recorded
4. Check the BSSID	Did the driver really connect to the AP we asked for?	reported as stage “bssid”
5. Check the IP layer	Done by l3.py (section 4): wait up to 4 s for a real address and gateway.	hand back to Windows
6. Clean up	Close RoamPredict's own old sockets (SocketRegistry.reset_all). Other programs are not touched.	
7. Measure	Connect, auth and IP times, success and fallback go into a RoamOutcome, then a SwitchRecord in SQLite.	

Security point: `params.strProfile = profile` (windows_wlan.py:529) passes only the **name** of the saved Windows profile. The password or certificate stays inside Windows; Python never sees it.

After the switch

The engine reports the result to the safety gate and the state machine. Success: COOLDOWN 20 s before any new switch. Failure: COOLDOWN 60 s, and after 2 failures in a row the kill switch blocks all switching until someone runs `roampredict unkill`. A suspected login prompt trips the kill switch immediately.

4. Act: checking the network layer (l3.py)

Being connected to Wi-Fi does not mean the internet works: the laptop also needs an IP address, a gateway and a correct ARP table. l3.py reads everything through Windows APIs instead of parsing command output, so it does not depend on the language of Windows.

Windows function	Used for	Admin rights
GetAdaptersAddresses	IPv4 address, prefix, gateway and DHCP server of the Wi-Fi adapter	no
IcmpSendEcho	Ping the gateway (only if the gateway answered ping before the switch)	no
FlushIpNetTable2	Clear the ARP cache of the Wi-Fi interface after a switch on the same network	yes
ipconfig /renew "Wi-Fi"	Renew DHCP when the network changed or there is no address	probably

Rules: same subnet and gateway → flush ARP; changed network or no address → renew DHCP. Our campus blocks ping, so the executor learns this before the switch and checks the address instead. A 169.254.x address means DHCP did not answer: there is no internet. Commands run with an argument list and an absolute path under %SystemRoot%\System32, never through a shell.

5. Act: talking to Windows in C (win32_structs.py)

wlanapi.dll and iphlpapi.dll are C libraries. Python calls them through `ctypes`, which needs every C structure rebuilt field by field: for example WLAN_CONNECTION_PARAMETERS (what WlanConnect receives), WLAN_BSS_ENTRY (one AP in a scan), IP_ADAPTER_ADDRESSES_LH (the adapter's addresses) and ICMP_ECHO_REPLY (a ping answer).

- Only fixed-width integer types are used, so every structure has exactly the Windows 64-bit layout on any OS.
- 30 tests check each structure's size and every field offset against the published Windows x64 layout.
- Each function also has declared argument types (PROTOTYPES in windows_wlan.py), so a wrong call raises an error instead of corrupting memory.

6. Remember: measuring internet outages (probe.py, latency.py)

With `run --probe www.google.com`, a background thread opens a TCP connection to www.google.com port 443 every 0.25 s (the name is looked up once, so DNS delays do not count). TCP is used because the campus blocks ping and some public addresses such as 1.1.1.1.

Class	File	Role
TcpPinger	latency.py	One timed TCP connect; counts why probes failed (timeout, refused, DNS…) in a Counter
OutageTracker	probe.py	Turns probe results into outages: 2 unanswered probes in a row = an outage, the first answer ends it
ConnectivityProbe	probe.py	Runs the probe in a daemon thread and queues NET_DOWN / NET_UP events for the engine
SocketRegistry	latency.py	Keeps RoamPredict's own sockets so they can be closed after a switch

Why 2 probes and not 1: single packets are lost on any busy network, so one lost probe would count false outages. An outage still running when the program stops is recorded up to that moment, so stopping never hides one.

7. Remember: the database (store.py)

One SQLite file, `%LOCALAPPDATA%\RoamPredict\roampredict.db`, opened only through `TopologyStore` (repository pattern). Queries are parameterised, the file uses WAL mode, and every row is returned as a Python dataclass.

Table	One row is	Used for
outages	one outage: start, duration, layer (wifi / internet), mode (dry-run / active), signal before	report: with vs without
switch_events	one roam, ours or Windows': times, success, requested and final BSSID, details	report: switch times
associations	a connection to an AP, with its source	“connected recently” score

Table	One row is	Used for
transitions	a learned move A → B with a count	“learned” score, WAITING
bss	every AP ever seen, with channel, band, 802.11r/k/v flags	doctor, report
meta	schema version	upgrades

Writes are validated: an outage with an unknown layer or mode, or a negative duration, is refused. `roampredict report --timeline` reads these tables to list outages and roams in time order.

8. Remember: the log file (telemetry.py)

Every event is written as one JSON line to `%LOCALAPPDATA%\RoamPredict\logs\roampredict.jsonl`, and the important ones are also shown as short lines on screen (START, STATE, DECISION, SWITCH_RESULT, NET_DOWN, STOP…). The file is only appended to, so it keeps the full history of every walk.

```
{"ts": "2026-10-07T16:44:16+0000", "kind": "switch_result",
 "msg": "OK: switched [never, connect 1914 ms (auth 773 ms), IP ready after 13 ms; fallback=none]",
 "success": true, "total_ms": 1914.0, ...}
```

- **Black box**: every bug we fixed was found by reading this file.
- **Privacy**: `scrub()` replaces any field whose name looks like a password, token, identity or certificate with “[removed]” before writing. No browsing data is recorded; everything stays on the laptop.
- **Thread-safe**: a lock protects the file because the probe thread and the main thread may both log.

9. Lists and dictionaries in Part 3

Where	Code	Why this structure
engine.py:200	session_outages = {"wifi": [], "internet": []}	Dictionary of lists: one list of outage durations per layer; count, sum and max give the STOP line
topology/store.py:484	groups.setdefault(key, []).append(rec)	Dictionary of lists: groups switches by who switched and AP class to compute median times
net/probe.py:71	self.outages: list[Outage] = []	List: outages in time order
net/latency.py:114	self.failures: Counter[str] = Counter()	Counter (a dictionary that counts): why probes were lost
telemetry.py:88	record = {"ts": ..., "kind": kind}	Dictionary turned into one JSON line
roam/executor.py:263	common = {"scanned": ..., "scan_ms": ..., "entries": ...}	Dictionary of shared fields unpacked into each Verification result

10. Field results and the bug we fixed

Walk	Mode	Internet lost	What Part 3 did
6 Oct, atelier, 5:41 (main laptop)	with	136 s (40%)	2 switches: 1 OK (connect 3.5 s, internet back 6.6 s after it died), 1 failed (0x38001), Windows took over
7 Oct, Aymane, first 2 min	with	93 s of 129 s (72%)	3 switches: 2 OK (1.9 s and 6.8 s), 1 connected but got no IP address
7 Oct, main laptop, 2 walks	without	331 s (92%) and 162 s (45%)	Measured only: shows how much Windows alone varies

The bug found in the field

On Aymane's laptop, the third switch connected to the new AP in 1.6 s, but DHCP never answered and Windows gave itself 169.254.173.113 (no internet). The old code reported the failure but stayed on that AP, which it had requested by BSSID. **Fix (commit fdfb6da)**: when the IP check fails after a switch, the executor now reconnects by profile name only (_hand_back), so Windows chooses the AP again and the one-BSSID request ends. Two tests reproduce the field case and fail without the fix.

11. Limits

- Every switch costs a few seconds, because WlanConnect drops the old link first (no make-before-break in Windows).
- Without administrator rights, ARP flush and DHCP renew are skipped.
- After a switch by BSSID, some drivers may keep the laptop on that AP; netsh wlan connect name="Campus Connecte" releases it.
- Part 3 cannot help while the laptop is disconnected or when Windows' 802.1X login keeps failing.
- Too few active walks to prove a general improvement; results must be compared on the same laptop and route.

12. Conclusion

Part 3 turns a decision into a safe action and turns every walk into data. Each switch is verified first, uses only the saved profile's name, is checked up to the IP layer and falls back to Windows when anything fails. Each outage is measured to a quarter of a second and stored with its mode, so walks with and without RoamPredict can be compared honestly. The field tests have already improved the code once; more walks on the same route are the next step to measure the real benefit.